

CS 3960 (3)

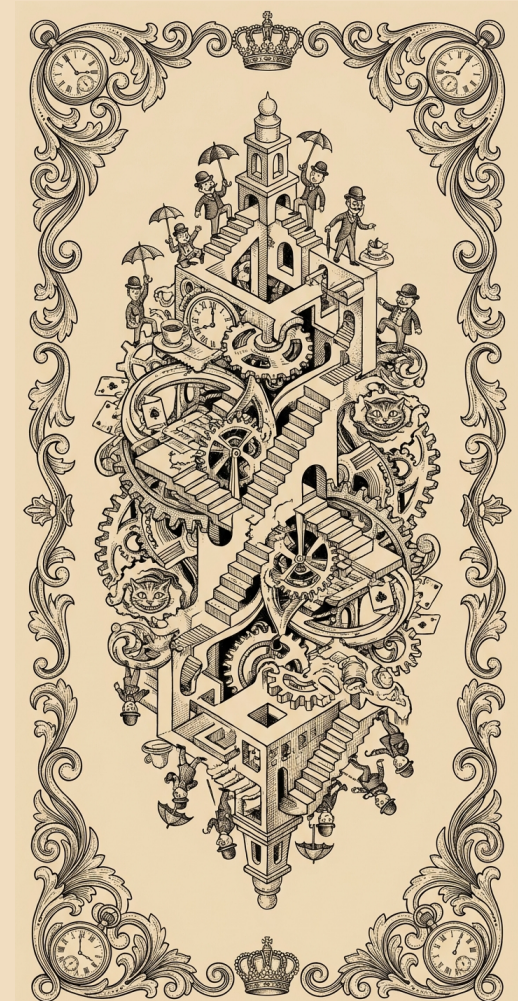
Vibe Coding

Spring 2026, *content* Pavel Panchekha and John Regehr, *illustrations* Gemini 3 Nano Banana Pro

- Reading assignment due Friday
 - <https://katalon.com/resources-center/blog/test-suite-management>
- How's the code review going?

There exist systems that are
effectively impossible to test.

Try to not create them.





Avoid test cases that...

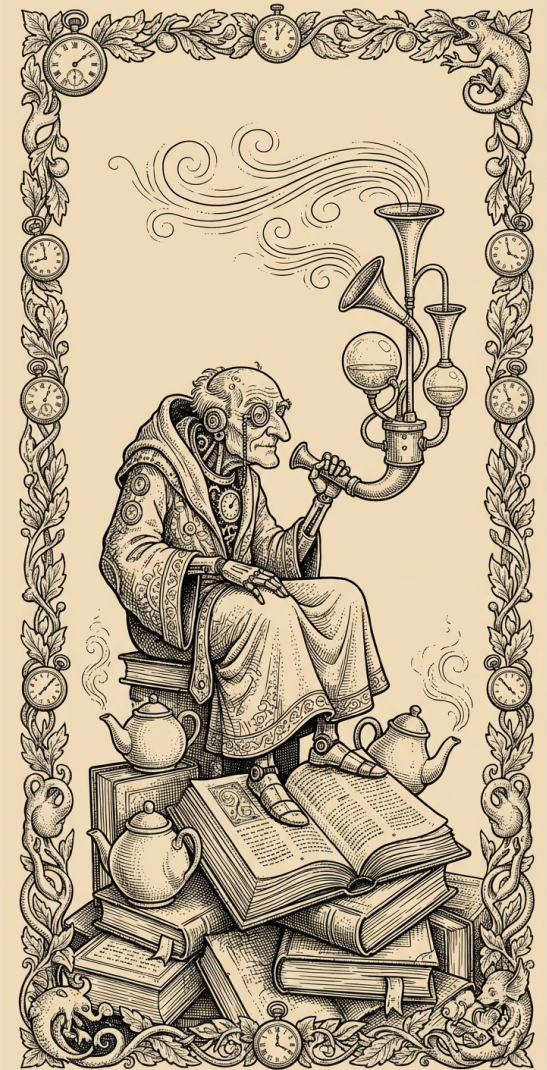
- Are not deterministic
- Are very slow to execute
- Don't have a specific purpose
- Don't elicit a behavior that we can agree is correct or incorrect

A **test oracle** tells us whether a test has passed or failed

The trivial oracle is a hard-coded answer

But many others are possible

Name a few

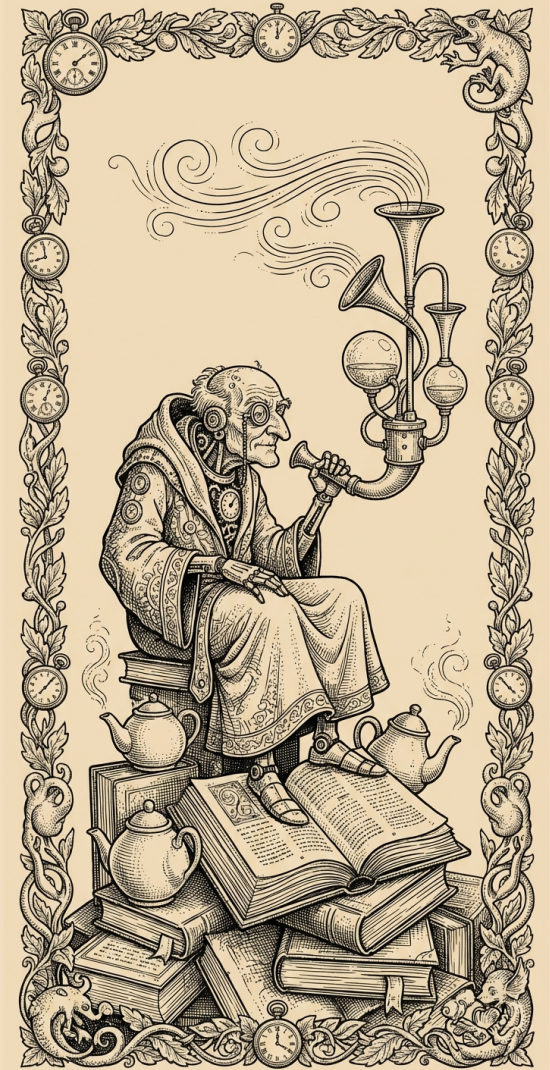


Some less obvious oracles...

- Language / environment rules
- Resource limits
- Internal consistency of the result
- Approximate bounds known by a human
- Differential — compare against a second implementation
- A specification — executable or otherwise

Creative and clever test oracles
can make or break a testing
effort

It's always worth asking: **What
kind of oracles could I use to
better test my software?**





Q: When do tests get run?

A: In general, the sooner the better.

The longer a bug lasts, the more expensive it is to fix.

Many project end up with a hierarchy of tests

- If you're modifying just one subsystem, then you start running the unit tests for that subsystem
- Then, the unit tests for the rest of the subsystems
- Finally, run integration tests



**How do we know if we have
enough tests?**

**How do we know if our tests
are good?**

Answer #1: Overkill

SQLite has 920 million lines of tests

But only 156 thousand lines of code

Let's look at how this works...





Answer #2: Mutation

The idea is cute: We treat the test cases as the thing that is being tested

We do this by introducing bugs into the system under test

What we want to see:

Any time we introduce a bug, one of our test cases catches it.

Can you see any issues with this plan?



Mutation testing is not super
widely used in practice

Answer #3: Code coverage

We look at what parts of the
system under test get executed
when we run the tests





What does it tell us about our test suite if some part of our code **is not** covered by any test?

What does it tell us about our test suite if some part of our code **is** covered by one or more tests?





“How to Misuse Code Coverage”
by Brian Marick is worth reading.

His point is that if you just blindly cover uncovered code, you’re doing it wrong.

Rather, think about why uncovered code existed in the first place.

Kinds of code coverage

- Function coverage
- Statement coverage
- Edge coverage
- Condition coverage





MC/DC coverage requires:

1. Each entry and exit point is invoked
2. Each decision takes every possible outcome
3. Each condition in a decision takes every possible outcome
4. Each condition in a decision is shown to independently affect the outcome of the decision.

Typically only used for safety-critical software

There are coverage metrics for
concurrent software, but none
are widely used

This seems like a bit of a
problem





In-class assignment:

Measure the total code coverage in percent that your editor's test suite gets, when testing your editor.

```
python3 -m pytest --cov=xxx
```

Report the % as the response to the Canvas quiz in the Week 5 module.